Compte rendu de la séance de travail

Aujourd'hui, nous avons poursuivi notre projet sur la transmission, le traitement et réception des trames sur les cartes STM32. Le travail s'est articulé autour de trois grandes phases : émission et réception sur une seule carte, génération de trames aléatoires, puis communication entre deux cartes distinctes.

1) Émission et réception de deux trames aléatoire sur une même carte

Dans un premier temps, nous avons travaillé sur **une seule carte STM32**, configurée pour transmettre deux trames fixes via **USART1** et les recevoir avec **UART5**.

Nous avons défini une structure de trame de **8 bits**, avec la répartition suivante :

- **3 bits d'identification**,
- **1 bit d'organisation**,
- **4 bits de données**.

Le **bit de parité** n'est **pas codé manuellement** : il a été **automatiquement géré par la configuration dans STM32CubeMX**, où nous avons activé la **parité paire** sur l'USART.

Ces deux trames sont générer de façon aléatoire ;

```
uint16_t trame1 = 0b11100000 | (rand() % 32);
uint16_t trame2 = 0b11100000 | (rand() % 32);
```

2) Réception des deux trames et traitement sur la même carte

Nous avons reçu deux trames via l'interface UART5, toujours sur la même carte STM32. Toutefois, pour des raisons de clarté, nous allons détailler le traitement d'une seule trame, le traitement de la seconde étant identique.

a) Suppression du bit de parité :

Contrairement à ce que l'on pourrait penser, le bit de parité, bien qu'ajouté automatiquement par l'USART lors de l'émission, n'est pas automatiquement supprimé à la réception. En effet, les trames sont reçues sur 9 bits (8 bits de données + 1 bit de parité), et il est donc nécessaire d'ignorer manuellement le bit de parité pour récupérer uniquement les 8 bits utiles.

Pour ce faire, on applique un masque binaire 0x00FF à la trame reçue afin de ne conserver que les 8 bits de données utiles, en éliminant le bit de parité : trame1 = trame_recue & 0x0FF;

Exemple : = trame_recue = 1011 0011 1000 0000 & 1111 1111 0000 0000 = $\overbrace{1011\ 0011}^{\text{Trame1}}$ 0000 0000

Le résultat cette opération nous donne l'octet complet **sans le bit de parité**, prêt à être traité.

b) Vérification de ID.

Avant de procéder au traitement, nous vérifions si la trame reçue est bien destinée à notre module. Pour cela, un identifiant (ID) a été défini au préalable. Nous extrayons les trois bits de poids fort de la trame à l'aide de l'opération suivante : ID = (trame1 >> 5) & 0b111;

Cette opération permet d'extraire l'ID contenu dans la trame. Si l'identifiant correspond à celui que nous avons prédéfini (if (ID == identifiant_attendu)), nous poursuivons le traitement. Dans le cas contraire, un message signalant un ID incorrect est affiché et la trame est ignorée.

```
else
{
  BSP_LCD_SetTextColor(LCD_COLOR_RED);
  BSP_LCD_DisplayStringAt(0, LINE(6), (uint8_t *)"TRAME1 ID ERROR", CENTER_MODE);
}
```

c) Récupération des 4 dernier bits qui constitue le une partie du message

Une fois l'identifiant vérifié, nous extrayons les 4 dernier bits de la trame, qui constituent la partie utile du message. Pour ce faire, nous définissons une variable et appliquons un masque binaire 0x0F afin de ne conserver que ces 4 bits : msg1 = data1 & 0x0F;

Explication de l'opération : data1 = 1011 1101 & 0000 1111 = 0000 1101

Ces 4 bits représentent une information codée que nous pourrons ensuite exploiter. Ensuite, nous appliquons exactement la même opération sur la seconde trame pour en extraire également les 4 bits utiles. Finalement, on obtient **msg1** et **msg2**, contenant chacun une partie du message. Ensemble, ils constituent le message complet, c'est-à-dire l'ensemble des informations transmises.

d) Organisation de la trame finale

L'ordre de concaténation des données dépend de la **valeur du bit d'organisation (le 4 -ème bit de chaque trame**). Ce bit sert à déterminer si l'information reçus (msg1 et msg2) doit être placée à gauche ou à droite dans le message final :

4 -ème bit1	4 -ème bit2	Action
1	0	msg1 à gauche, msg2 à droite
0	1	msg2 à gauche, msg1 à droite
0	0	Erreur
1	1	Erreur

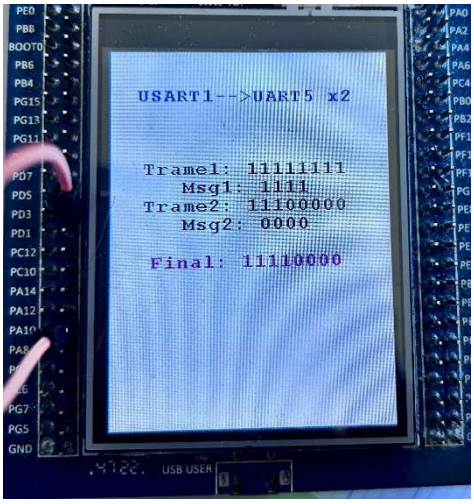
```c
if ((ID1 == identifiant_attendu) && (ID2 == identifiant_attendu))
{
  uint8_t b4_msg1 = (data1 >> 4) & 0x01;
  uint8_t b4_msg2 = (data2 >> 4) & 0x01;

  if ((b4_msg1 == 1 && b4_msg2 == 1) || (b4_msg1 == 0 && b4_msg2 == 0))
  {
    BSP_LCD_SetTextColor(LCD_COLOR_RED);
    BSP_LCD_DisplayStringAt(0, LINE(11), (uint8_t *)"ERREUR MSG FINAL", CENTER_MODE);
  }
  else
  {
    if (b4_msg1 == 1 && b4_msg2 == 0)
    {
      // msg1 à gauche, msg2 à droite
      memcpy(MSG_F, bits4_1, 4);
      memcpy(MSG_F + 4, bits4_2, 4);
    }
    else if (b4_msg1 == 0 && b4_msg2 == 1)
    {
      // msg2 à gauche, msg1 à droite
      memcpy(MSG_F, bits4_2, 4);
      memcpy(MSG_F + 4, bits4_1, 4);
    }

    MSG_F[8] = '\0';
    sprintf(texte, "Final: %s", MSG_F);
    BSP_LCD_SetTextColor(LCD_COLOR_MAGENTA);
    BSP_LCD_DisplayStringAt(0, LINE(11), (uint8_t *)texte, CENTER_MODE);
  }
}

HAL_Delay(1000);
```

3) Résultat au rendez-vous



4) Communication entre deux cartes distinctes

Dans cette partie, nous avons configuré notre carte STM32 pour qu'elle puisse recevoir des données via l'interface UART5, en utilisant la broche PD2 comme entrée. Pour cela, nous avons dû désactiver l'émission sur UART1, car elle n'était plus nécessaire à ce stade. Afin de tester la

communication, nous avons utilisé le code d'un camarade qui émet deux trames à l'aide d'un Arduino. Grâce à cette émission fiable, nous avons pu recevoir correctement les deux trames sans fil sur notre carte STM32.

En revanche, nous avons rencontré des difficultés lors de l'émission depuis une autre carte STM32. La transmission n'était pas stable ou ne fonctionnait pas comme prévu, ce qui a limité notre capacité à tester un échange complet entre deux STM32